ENS 492 – Graduation Project (Implementation)

Progress Report II

Project Title:

Implementing Deep Neural Networks (DNNs) Using GPUs

Group Members:

Ada Boran Yılmaz (30789)

Enes Çağlar (31109)

Supervisor(s):

Ömer Ceylan

Date:

15/11/2025

Table of Contents

1. PROJECT SUMMARY

This project focuses on how to speed up deep neural networks, particularly smaller ones like MobileNet and EfficientNet, by improving how they use GPU resources. Our objective is to achieve real-time inference on edge devices, which inherently possess limited computational power, memory, and energy budgets. To achieve this, we're diving into NVIDIA's parallel processing with low-level CUDA, using TensorRT for optimization, and performing deep kernel-level tuning of critical operations such as convolutions and matrix multiplications.

The main issue we're trying to solve is the limited accessibility and optimization of GPU implementations for deep learning inference in real-world edge settings. While there are domain-specific accelerators like TPUs and Jetson devices, these are expensive, closed-source, and usually inflexible. On the other hand, general purpose GPUs are widely available, but there are not enough optimized, reproducible, and well documented CUDA implementations that practitioners can directly adapt for edge or near edge deployment. This creates a gap between academic models and deployable, efficient inference pipelines on commodity GPUs. Therefore, we seek to offer a software-only, open, and efficient GPU optimization framework that will make AI inference environmentally sustainable and economically viable for deployment.

Our motivation is to make high-performance inference achievable without dedicated hardware accelerators. In this work, we target a reproducible GPU optimization workflow, finding the right balance between performance and resource consumption by evaluating raw CUDA, TensorFlow/PyTorch GPU acceleration, and TensorRT-based deployment. A fast CUDA/TensorRT inference pipeline with the goal of running DNNs on regular GPUs represents the final deliverable.

The objectives of this capstone design project are to analyze and understand the computational requirements of lightweight DNNs, to implement selected models on CPU, standard GPU frameworks (TensorFlow/PyTorch), and raw CUDA, and to construct an optimized inference pipeline using CUDA and TensorRT that runs efficiently on general purpose GPUs.

Our design process follows the fundamental elements of engineering design. For objectives and criteria, we define measurable performance criteria such as accuracy, power usage, memory footprint, latency, and throughput for real time edge style workloads. For synthesis, we synthesize techniques from the literature and existing frameworks such as memory coalescing, kernel fusion, mixed precision inference into a coherent optimization strategy. For analysis, we profile and analyze CPU, baseline GPU, and optimized CUDA/TensorRT implementations to understand bottlenecks in computation, memory access, and data movement. For construction, we construct CUDA kernels and TensorRT deployment configurations, integrating them into a complete inference pipeline that can be executed on commodity GPUs. For testing and evaluation, we rigorously test and evaluate the system under realistic workloads, comparing different implementations in terms of latency, accuracy, power usage, and scalability.

The intended result of the project is reproducible GPU optimization workflow and a fast CUDA/TensorRT based inference pipeline for lightweight DNNs, ready for deployment on regular GPUs rather than specialized accelerators. The design explicitly adheres to realistic constraints such as limited computational resources on edge like setups, economic feasibility by reusing existing GPU infrastructure instead of purchasing specialized hardware, environmental impact reducing energy consumption per inference and scalability allow the same optimization framework to be extended to other models and datasets. By the end of the project, we aim to provide not only an optimized implementation, but also a documented set of guidelines and empirical results that demonstrate how well the proposed CUDA/TensorRT pipeline meets the defined performance and efficiency requirements for real time edge deployment.

2. SCIENTIFIC/TECHNICAL DEVELOPMENTS

Since the first progress report, we have made notable technical improvements in both understanding theoretical parts of the project and implementing practical phases. The main improvements that we've achieved since our last report are GPU research, environmental setup, deep learning model benchmarking, and optimization research with NVIDIA TensorRT and NVIDIA Nsight.

We conducted deep research on GPU architecture and performance profiling tools to evaluate and optimize performance of our models. We specifically focused on NVIDIA Tesla A100 GPUs which are available on the TRUBA (Türk Ulusal Bilim e-Altyapısı) server's high performance computing cluster. This research included understanding the internal structure of GPUs by focusing on how computations are distributed across grids, thread blocks, warps, and CUDA cores. We studied concepts of the gigathread engines, streaming multiprocessors, warps, warp scheduling, and tensor cores to learn how GPUs execute computations in parallel and what factors affect performance of these computations in detail.

Then, we conducted practical work on TRUBA to leverage its high computing clusters with GPUs to infer our models. We made research on Akya-CUDA and Barbun-CUDA clusters to see which one should we use to infer our models and we decided on Akya-CUDA because of it has better GPUs compared to Barbun-CUDA and availability of nodes in Akya-CUDA are better compared to Barbun-CUDA. Akya-CUDA has NVIDIA Tesla V100 GPUs and Barbun-CUDA has NVIDIA Tesla P100 GPUs, V100 offers significantly higher computational power and efficiency compared to P100. Since V100 has 40% more CUDA cores (5120 vs. 3584), 640 Tensor Cores for mixed-precision deep learning acceleration (P100 do not have Tensor Cores), and higher memory bandwidth (900.1 GB/s vs. 720.9 GB/s), it is much better suited for DNN inference and optimization tasks.

We benchmarked pretrained MobileNet and EfficientNet models in both TensorFlow and PyTorch, ran them under three configurations:

Min: 1 GPU / 16 GB RAM / 10 CPU

Average: 2 GPUs / 32 GB RAM / 20 CPU

Max: 4 GPUs / 64 GB RAM / 40 CPU

The results showed that PyTorch achieved up to 3.5× faster inference throughput than TensorFlow at smaller batch sizes (e.g., batch 16), while TensorFlow outperformed PyTorch at larger batches (≥128) and average configuration achieved highest throughput across most batch sizes, which shows the most balanced and efficient use of computational resources can be better compared to min and max.

Then, we conducted research on how to improve performance and optimize our models. We chose the NVIDIA TensorRT tool to optimize model performance and explored it in detail. We studied horizontal and vertical diffusion techniques, where a combination of multiple sequential or parallel layers to minimize memory transfers, and adding a concatenation layer to influence memory operations without actual calculations. Also, we learned how to analyze kernel activities, measure memory bandwidth, and identify performance bottlenecks like underutilized streaming multiprocessors and excessive global memory access by using NVIDIA Nsight. We are going to use these profiling studies by developing C++ implementation of our neural network with raw CUDA kernels, where we plan to apply optimization strategies like kernel fusion, shared memory utilization, and TensorCore acceleration.

Building upon these developments, we deepened our architectural analysis of GPU microarchitectures by comparing modern NVIDIA families such as Ampere and Hopper. This allowed us to understand how next generation features such as FP8/FP16/TF32 throughput characteristics, Tensor Core generational differences, and high bandwidth memory technologies like HBM2e and HBM3 directly influence inference performance and model scalability. This analysis clarified which hardware characteristics our custom kernels should target and highlighted architectural constraints such as memory-bandwidth ceilings, warp-level execution

behavior, and tensor core utilization pathways. These insights formed the conceptual foundation that guides all subsequent optimization decisions.

During the preliminary design stage, we synthesized our theoretical research into a functional architectural layout for the complete inference pipeline. We delineated the full data trajectory, extending from input preprocessing and device memory protocols to model execution, while simultaneously defining how the CPU would regulate GPU kernel launches and memory transfers. Concurrently, we developed structural diagrams for the C++ inference engine to specifically identify which layers would rely on existing frameworks and which were candidates for replacement with custom CUDA kernels. These early designs set up a baseline forward pass and laid the essential foundation needed for our later optimization work.

Our early experiments and profiling results helped us lock in the key design decisions that are now steering the project. We established a workflow that relies strictly on hard data, meaning specific kernel metrics tell us exactly what to optimize rather than just guessing based on intuition. This approach compelled us to focus first on operations that display excessive memory traffic or poor multiprocessor utilization. Furthermore, we chose to integrate TensorRT as a comparative baseline to rigorously evaluate the efficacy of our manual optimizations and established mixed precision execution as a central component of the final strategy. The choice to use PyTorch for prototyping was driven by its superior performance with small batch sizes and the relative ease of integrating custom CUDA extensions. These decisions guarantee that our detailed design phase rests on concrete empirical data and stays fully aligned with the overarching objectives of the project.

During the detailed design stage, we sharpened the blueprint for the most computationally demanding components of the model. We explicitly mapped out the block and grid configurations for our custom convolution kernels and established a shared memory tiling strategy designed to curb global memory latency. We also defined the precise mechanism for fusing intermediate activations into individual CUDA kernels to effectively minimize launch overhead. Furthermore, we structured the specific steps needed to translate the forward pass into a standalone C++ execution pipeline capable of hosting both vendor optimized libraries and our bespoke implementations. These comprehensive specifications provide a direct roadmap for

constructing the optimized kernels and guarantee that our development work systematically targets the bottlenecks revealed through profiling.

Since the last report, we also examined the potential for commercialization and the associated Freedom-to-Use considerations. Lightweight GPU-optimized inference pipelines have growing relevance in IoT analytics, robotics, and embedded AI systems domains where cost-effective, software-based acceleration is preferred over proprietary hardware solutions. Our review of existing patents and commercial accelerators indicates that the techniques we rely on such as manual CUDA kernel development, shared memory tiling, and mixed precision execution do not conflict with restricted IP, as they are general optimization practices rather than patented algorithms. Furthermore, NVIDIA's CUDA, cuBLAS, cuDNN, and TensorRT libraries are licensed for commercial use, which minimizes FTU risks. This positions the project as a viable foundation for an open, reproducible, and deployment-ready inference optimization workflow that could be extended into an industry-facing toolkit.

3. ENCOUNTERED PROBLEMS

This segment evaluates the current standing of the project relative to the initial timeline while addressing specific challenges and strategic adjustments. The implementation phase is proceeding strictly according to plan, maintaining adherence to the established schedule without notable delays. We have now finalized the foundational stages of the work, which encompassed a thorough literature review, an extensive analysis of GPU architectures, and the complete configuration of the TRUBA Akya CUDA environment. As this foundational stage was completed without issue, our original project objectives remain entirely relevant and have not required any modification. Our main goal remains the implementation and optimization of lightweight DNNs on GPUs using C++/CUDA.

Matching our progress to the original timeline has allowed us to avoid taking any major corrective steps. The difficulties that have arisen were the predictable technical issues, inherent to the nature of the research and development process. Getting the TRUBA Akya-CUDA development environment configured took a considerable, though planned, block of time. That work focused on managing software dependencies to get the NVIDIA V100 drivers, specific CUDA toolkit versions, and the initial benchmarking frameworks operating correctly together. Concurrently, a significant time investment was dedicated to the comparative analysis of TRUBA clusters and a deep-dive investigation into the V100 GPU architecture, which was essential foundational research to inform subsequent C++ implementation and optimization strategies.

The preliminary benchmarks discussed in Section 2 didn't require us to alter the project's overall strategy. If anything, they confirmed we're on the right track with our methods. Seeing the performance gaps between TensorFlow and PyTorch at different batch sizes proved to us that switching to a C++/TensorRT implementation is the right move. We're expecting that this will give us a consistent performance, regardless of which high-level framework we start with. Our framework tests will now be our baseline, which will give us a solid point of comparison to measure exactly how much speed we gain from our custom C++ and low-level optimizations.

As of this report, no major changes have been made to the original project plan or timeline. All milestones for Weeks 1-7 literature review, GPU architecture study, environment

setup, TRUBA benchmarking, and preliminary profiling—have been completed as scheduled. Since our conceptual and preliminary design phases concluded on time, there was no need to adjust deliverables or shift tasks into later weeks.

Although the environment setup required more effort than initially expected, these challenges were fully absorbed within the original time allocation. To avoid future schedule pressure, we prepared an updated micro-timeline for the next phase, where CUDA kernel development and TensorRT integration will run in parallel rather than sequentially. This adjustment is not a corrective action for delays but a preventive strategy to maintain high momentum in the detailed design phase.

The absence of major delays has allowed us to proceed exactly as envisioned in the proposal. Furthermore, the additional architectural knowledge and profiling insights gained during this period are expected to accelerate the kernel-implementation stage rather than slow it down. Because we now have concrete performance baselines and a fully configured development environment, the next stages—raw CUDA implementation, kernel fusion, Tensor Core integration, and C++ pipeline construction—can begin without uncertainty or structural modifications.

4. TASKS TO BE COMPLETED BEFORE FINAL REPORT

In the remaining phase of the project, the focus will shift from architectural research and benchmarking to the construction and optimization of the full CUDA based inference pipeline. Following the capstone design process, the remaining tasks span the detailed design, construction, testing, and evaluation phases. These tasks will prepare the system for final benchmarking, documentation, and adress the final presentation requirements.

In the detailed design phase during the weeks 8 and 9, we are going to finalize the block and grid configurations for all custom CUDA kernels, finalize the shared memory tiling layout for depthwise and pointwise convolutions, define the Tensor Core enabled execution paths (FP16 kernels) and complete integration design for C++ host code and CUDA device functions.

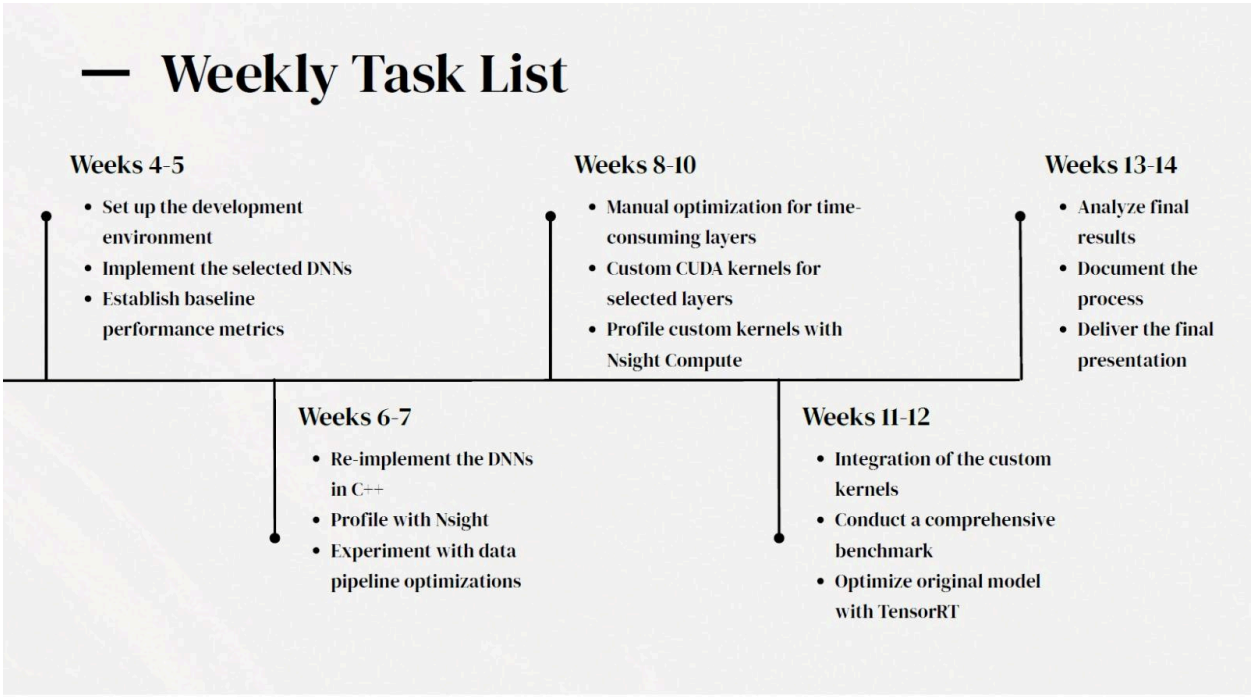
During weeks 9 and 11, we are going to implement the C++ forward pass for MobileNet/EfficientNet using raw CUDA kernels, implement kernel fusion for expand depthwise and project layers, integrate the custom kernels into the C++ execution pipeline, install and configure TensorRT and ensure interoperability with the C++ codabase.

During weeks 11 and 12, we are going to run Nsight Systems and Nsight Compute profiling sessions on TRUBA, identify bottlenecks such as warp divergence, memory stalls, or low SM utilization, apply optimizations iteratively based on profiling feedback, validate correctness of optimized kernels by comparing outputs with PyTorch/TensorFlow baselines.

During weeks 12 and 13, we are going to conduct full end to end benchmarks of PyTorch inference, TensorFlow inference, TensorRT optimized inference, custom CUDA/C++ inference, and measure final metrics latency, throughput, memory consumption, and SM utilization, document differences between baseline and optimized performance.

During weeks 13 and 14, we are going to prepare the final project report including full benchmark tables and profiling insights, produce the final presentation detailing methodology, CUDA kernel design, and evaluation outcomes, and prepare reproducible instructions for running the C++/CUDA implementation on TRUBA.

Table 1: Weekly Task List



5. REFERENCES

Abadi, M., et al. (2016). TensorFlow: Large-scale machine learning on heterogeneous systems. https://www.tensorflow.org/

GPU [Presentation]. Canva. https://www.canva.com/design/DAG2WURUo_4/CjvscaZEYAThGbX2H9bZ4g/edit?utm_content=DAG2WURUo_4&utm_campaign=designshare&utm_medium=link2&utm_source=sharebutton

Howard, A. G., et al. (2017). MobileNets: Efficient convolutional neural networks for mobile vision applications. arXiv preprint arXiv:1704.04861. https://arxiv.org/abs/1704.04861

NVIDIA Corporation. (2024). CUDA C++ programming guide. NVIDIA Developer. https://docs.nvidia.com/cuda/cuda-c-programming-guide/

NVIDIA Corporation. (2024). NVIDIA Nsight systems user guide. NVIDIA Developer. https://developer.nvidia.com/nsight-systems

NVIDIA Corporation. (2024). NVIDIA TensorRT developer guide. NVIDIA Developer. https://developer.nvidia.com/tensorrt

Paszke, A., et al. (2019). PyTorch: An imperative style, high-performance deep learning library. Advances in Neural Information Processing Systems, 32. https://pytorch.org/

TensorRT [Presentation]. Canva. https://www.canva.com/design/DAG2_GkTZ_o/Q-ndD2791GyiqH3rxMTzQQ/edit?utm_content=DAG2_GkTZ_o&utm_campaign=designshare&utm_medium=link2&utm_source=sharebutton

Truba Server [Presentation]. Canva. https://www.canva.com/design/DAG3pASbkfk/UZJr5b4b635UcXRMnGvrXA/edit?utm_content=DAG3pASbkfk&utm_campaign=designshare&utm_medium=link2&utm_source=sharebutton

TÜBİTAK ULAKBİM. (2024). TRUBA user guide. Türk Ulusal Bilim e-Altyapısı (TRUBA). https://docs.truba.gov.tr